

UNIVERSIDAD TECNOLÓGICA DE LOS ANDES



Escuela Profesional de Ingeniería De Sistemas e Informática Filial Cusco



Curso:

Computación Gráfica

Docente:

Mag. Elvio Tintaya Zegarra

Investgación:

El uso de las siguientes herramientas en Python

Presentada por:

Huaman Ataulluco Ricardo Manuel

Cusco – 2026 – I

Dedicatoria

Este trabajo se la dedicó a Dios y a mis seres queridos que siempre me apoyan y me exigen cada día para poder lograr un futuro brillante con muchas cualidades destacables en nuestro entorno, De manera especial ,a mis padres que gracias a su sacrificio y esfuerzo hacen posible que pueda alcanzar mis sueños.

ÍNDICE

INTRODUCCIÓN.....	5
CAPÍTULO I	6
BLOQUE GRÁFICOS Y ANIMACIÓN EN PYTHON.....	6
1.1 Concepto.....	6
1.2 Matplotlib	6
1.2.1 Código (Google Colab): trayectorias de proyectiles.....	6
1.2.2 Ejecución (captura).....	7
1.3 Matplotlib.animation / FuncAnimation.....	7
1.3.1 Código (Google Colab): punto que recorre una espiral	7
1.3.2 Ejecución (captura).....	9
1.4 NumPy (física básica para animación).....	9
1.4.1 Código (Google Colab): camino aleatorio 2D	9
1.4.2 Ejecución (captura).....	10
1.5 Plotly (animación interactiva de trayectorias)	10
1.5.1 Código (Google Colab): punto animado sobre una onda seno	10
1.5.2 Ejecución (captura).....	11
1.6 Pygame (mini videojuegos y animación 2D).....	11
1.6.1 Código (Python local): bala que rebota.....	11
1.6.2 Ejecución (captura).....	13
1.7 MatManim (animaciones matemáticas)	13
1.7.1 Código de escena (ejecución con manim).....	13
1.7.2 Ejecución (captura).....	14
1.8 VPython (animación 3D sencilla)	14
1.8.1 Código (Python/Glowsript): órbita de un satélite	14
1.8.2 Ejecución (captura).....	15
CAPÍTULO II	16
BLOQUE ANÁLISIS DE DATOS Y APRENDIZAJE AUTOMÁTICO	16
2.1 Concepto general del bloque.....	16
2.2 Pandas 16	
2.2.1 Código (Google Colab): resumen de ventas por categoría	16
2.2.2 Ejecución (captura).....	17
2.3 NumPy (bloque de datos)	17

2.3.1	Código (Google Colab): matriz y estadísticas	17
2.3.2	Ejecución (captura).....	17
2.4	Matplotlib (visualización de datos).....	18
2.4.1	Código (Google Colab): dispersión con tendencia	18
2.4.2	Ejecución (captura).....	19
2.5	Seborn	19
2.5.1	Código (Google Colab): matriz de correlación	19
2.5.2	Ejecución (captura).....	20
2.6	Plotly (visualización interactiva en análisis de datos)	20
2.6.1	Código (Google Colab): barras interactivas	20
2.6.2	Ejecución (captura).....	21
2.7	SciPy.....	21
2.7.1	Código (Google Colab): regresión lineal simple	21
2.7.2	Ejecución (captura).....	22
2.8	Scikit-learn	22
2.8.1	Código (Google Colab): clasificación binaria	22
2.8.2	Ejecución (captura).....	23
2.9	Streamlit23	
2.9.1	Código: mini dashboard de serie temporal	23
2.9.2	Ejecución (captura).....	24
2.10	Bokeh 24	
2.10.1	Código (Google Colab / HTML): línea interactiva.....	24
2.10.2	Ejecución (captura).....	25
CONCLUSIONES.....		26
BIBLIOGRAFÍA.....		27

INTRODUCCIÓN

El ecosistema científico de Python se ha consolidado alrededor de bibliotecas que proporcionan estructuras de datos multidimensionales, algoritmos numéricos y herramientas de visualización, lo que facilita la implementación de experimentos computacionales en ingeniería y ciencia de datos. Harris et al. (2020) destacan que NumPy se ha convertido en la base de este ecosistema al introducir un modelo de programación con arreglos que simplifica el tratamiento de datos y cálculos intensivos.

Por su parte, Hunter (2007) describe Matplotlib como un entorno de gráficos 2D capaz de generar figuras de calidad de publicación para distintas aplicaciones científicas, lo que explica su adopción como componente central en numerosas herramientas de visualización.

Sobre estos cimientos se han desarrollado bibliotecas especializadas para gráficos 2D y 3D, animaciones educativas, análisis estadístico, aprendizaje automático y visualización web interactiva. Entre ellas destacan Matplotlib.animation, Plotly, Pygame, Manim y VPython para la generación de visualizaciones y simulaciones, así como Pandas, Seaborn, SciPy, Scikit-learn, Streamlit y Bokeh para la limpieza, el análisis y la comunicación de resultados.

En este contexto, el objetivo del presente trabajo es investigar el uso de estas bibliotecas, presentar ejemplos de código no genéricos que ilustren su aplicación en animaciones y análisis de datos, y documentar los resultados siguiendo las normas de citación de la séptima edición de APA.

CAPÍTULO I

BLOQUE GRÁFICOS Y ANIMACIÓN EN PYTHON

1.1 Concepto

El bloque de gráficos y animación se centra en bibliotecas que permiten representar visualmente funciones, trayectorias y escenas interactivas en 2D y 3D. Según Harris et al. (2020), NumPy proporciona arreglos y operaciones vectorizadas que sirven de base para generar datos numéricos que luego se visualizan con otras bibliotecas. Hunter (2007) caracteriza a Matplotlib como “un entorno de gráficos 2D” para Python, diseñado tanto para scripts interactivos como para generar figuras listas para publicación. A partir de estos cimientos, herramientas como Plotly, Pygame, Manim y VPython amplían el alcance hacia animaciones interactivas en la web, videojuegos educativos, animaciones matemáticas y simulaciones físicas en tres dimensiones (Plotly Technologies Inc., 2015; Pygame Community, 2024; Manim Community Developers, 2024; VPython Developers, 2020)

1.2 Matplotlib

Hunter (2007) presenta Matplotlib como un paquete de gráficos 2D para Python que permite crear figuras de alta calidad para aplicaciones científicas y técnicas. El autor indica que la biblioteca ofrece una interfaz similar a MATLAB, capaz de producir gráficos de líneas, dispersión, barras e histogramas, con un control detallado sobre estilos, ejes y formatos de salida. En su artículo se señala que Matplotlib “provide[s] a flexible, cross-platform plotting library for Python” (Hunter, 2007), lo que ha contribuido a su adopción como estándar de facto en muchos proyectos de ingeniería.

1.2.1 Código (Google Colab): trayectorias de proyectiles

```
# @title Trayectorias de tres proyectiles con Matplotlib
import numpy as np
import matplotlib.pyplot as plt

g = 9.8
v0 = 25.0
angulos = [30, 45, 60]

t = np.linspace(0, 5, 200)

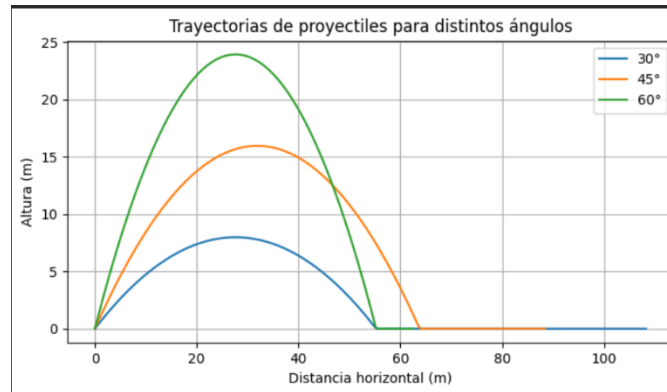
plt.figure(figsize=(8, 4))

for ang in angulos:
    theta = np.radians(ang)
    x = v0 * np.cos(theta) * t
    y = v0 * np.sin(theta) * t - 0.5 * g * t**2
    y = np.maximum(y, 0)
    plt.plot(x, y, label=f'{ang}°')

plt.title("Trayectorias de proyectiles para distintos ángulos")
plt.xlabel("Distancia horizontal (m)")
```

```
plt.ylabel("Altura (m)")
plt.grid(True)
plt.legend()
plt.show()
```

1.2.2 Ejecución (captura)



El programa calcula y grafica tres trayectorias de proyectiles con igual velocidad inicial pero diferentes ángulos de lanzamiento. Cada curva permite comparar visualmente la altura máxima y el alcance horizontal alcanzado, lo que ilustra el efecto del ángulo sobre el movimiento parabólico en dos dimensiones.

1.3 Matplotlib.animation / FuncAnimation

La documentación de Matplotlib explica que el submódulo `matplotlib.animation` permite construir animaciones a partir de secuencias de cuadros, actualizando progresivamente los elementos de una figura (Matplotlib Development Team, s. f.). Dentro de este submódulo, la clase `FuncAnimation` está diseñada para llamar repetidamente a una función de actualización que modifica los datos de uno o varios artistas, controlando el número de cuadros y el intervalo de tiempo entre ellos. De acuerdo con la guía de animación, `FuncAnimation` “creates animations by repeatedly calling a function *func*”, lo que la hace adecuada para representar movimientos continuos en gráficos científicos (Matplotlib Development Team, s. f.).

1.3.1 Código (Google Colab): punto que recorre una espiral

Animación de un punto en espiral con FuncAnimation (versión Colab-friendly)

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from matplotlib import rc
from IPython.display import HTML

# Configuración especial para que la animación se vea en Colab
rc('animation', html='jshtml')

# Datos de la espiral
t = np.linspace(0, 8 * np.pi, 400)
```

```

r = np.linspace(0.1, 4, 400)
x = r * np.cos(t)
y = r * np.sin(t)

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(-4.5, 4.5)
ax.set_ylim(-4.5, 4.5)
ax.set_aspect("equal")
ax.grid(True)

# Línea de trayectoria completa (gris) y punto rojo
trayectoria, = ax.plot([], [], color="lightgray")
punto, = ax.plot([], [], "o", color="red")

def init():
    # Secuencias vacías al inicio
    trayectoria.set_data([], [])
    punto.set_data([], [])
    return trayectoria, punto

def update(frame):
    # Hasta el frame actual, dibujamos la parte recorrida
    trayectoria.set_data(x[:frame], y[:frame])

    # Para el punto, pasamos arrays de longitud 1: [x_val], [y_val]
    punto.set_data([x[frame]], [y[frame]])

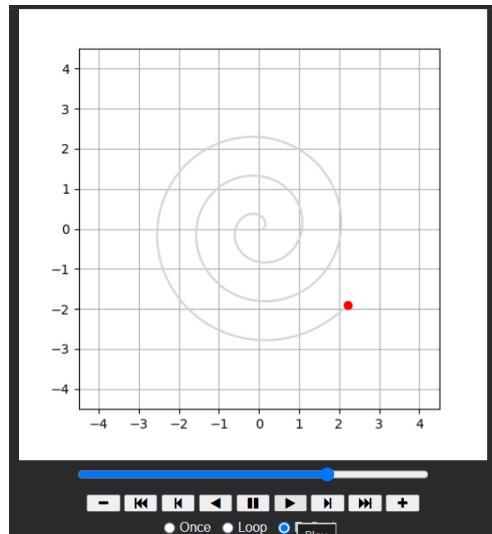
    return trayectoria, punto

ani = FuncAnimation(
    fig, update,
    frames=len(t),
    init_func=init,
    interval=20,
    blit=True
)

HTML(ani.to_jshtml())

```


1.3.2 Ejecución (captura)



El programa calcula y grafica tres trayectorias de proyectiles con igual velocidad inicial pero diferentes ángulos de lanzamiento. Cada curva permite comparar visualmente la altura máxima y el alcance horizontal alcanzado, lo que ilustra el efecto del ángulo sobre el movimiento parabólico en dos dimensiones.

1.4 NumPy (física básica para animación)

Harris et al. (2020) describen NumPy como la biblioteca fundamental de programación con arreglos en Python, al proporcionar estructuras de datos multidimensionales y operaciones vectorizadas eficientes. Los autores destacan que NumPy “provides an essential foundation for scientific computing in Python”, por su papel como base de bibliotecas de análisis, visualización y *machine learning*. En el contexto de la física básica, sus funciones permiten calcular posiciones, velocidades y trayectorias de forma compacta, lo que facilita generar datos para animaciones y simulaciones.

1.4.1 Código (Google Colab): camino aleatorio 2D

```
# @title Camino aleatorio 2D con NumPy
import numpy as np
import matplotlib.pyplot as plt

n = 500

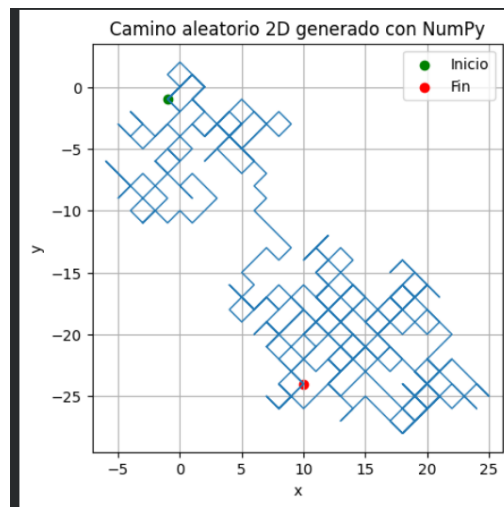
pasos_x = np.random.choice([-1, 1], size=n)
pasos_y = np.random.choice([-1, 1], size=n)

x = np.cumsum(pasos_x)
y = np.cumsum(pasos_y)

plt.figure(figsize=(5, 5))
plt.plot(x, y, linewidth=1)
plt.scatter([x[0]], [y[0]], color="green", label="Inicio")
plt.scatter([x[-1]], [y[-1]], color="red", label="Fin")
```

```
plt.title("Camino aleatorio 2D generado con NumPy")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.axis("equal")
plt.show()
```

1.4.2 Ejecución (captura)



Este script construye un paseo aleatorio bidimensional en el que cada paso desplaza la posición una unidad hacia la izquierda o derecha y arriba o abajo. El uso de arreglos y sumas acumuladas de NumPy permite generar la trayectoria completa con pocas instrucciones, que luego se visualiza en un plano mediante Matplotlib.

1.5 Plotly (animación interactiva de trayectorias)

La documentación oficial de Plotly para Python presenta la biblioteca como una solución de código abierto para crear gráficos interactivos en la web a partir de datos científicos y de negocio (Plotly Technologies Inc., 2015). Según esta documentación, Plotly permite generar figuras con zoom, desplazamiento, cuadros de información y animaciones, integrándose fácilmente con Jupyter y pandas. Esta combinación la hace adecuada para construir animaciones de trayectorias y datos en movimiento que pueden explorarse directamente desde el navegador.

1.5.1 Código (Google Colab): punto animado sobre una onda seno

```
# @title Animación interactiva de un punto sobre una onda seno con Plotly
import numpy as np
import pandas as pd
import plotly.express as px

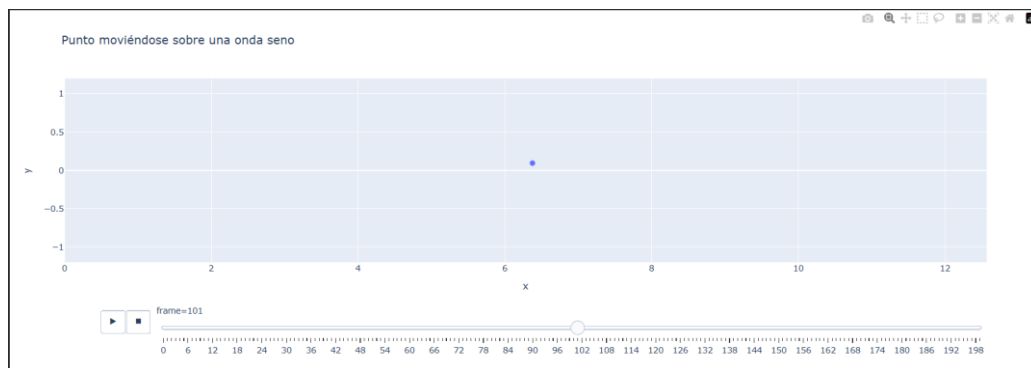
x = np.linspace(0, 4 * np.pi, 200)
y = np.sin(x)
```

```
df = pd.DataFrame({
    "x": x,
    "y": y,
    "frame": np.arange(len(x))
})

fig = px.scatter(
    df, x="x", y="y",
    animation_frame="frame",
    range_x=[0, 4 * np.pi],
    range_y=[-1.2, 1.2],
    title="Punto moviéndose sobre una onda seno"
)

fig.update_traces(marker=dict(size=8, color="red"))
fig.show()
```

1.5.2 Ejecución (captura)



El programa define una onda seno y asocia cada punto a un índice de cuadro de animación. Plotly utiliza ese índice para mover un punto a lo largo de la curva, generando una animación interactiva con controles de reproducción que permiten pausar, retroceder y avanzar la trayectoria directamente en el entorno de Colab.

1.6 Pygame (mini videojuegos y animación 2D)

La documentación oficial describe Pygame como un conjunto de módulos de Python basados en SDL, diseñado para desarrollar videojuegos y aplicaciones multimedia portables entre sistemas operativos (Pygame Community, 2024). En la guía para principiantes se indica que Pygame proporciona “modules designed for writing video games”, incluyendo funciones para inicializar ventanas, dibujar superficies, manejar eventos de teclado y ratón, y actualizar la pantalla en un bucle de juego. Estas características permiten implementar animaciones 2D con objetos en movimiento, colisiones y control en tiempo real.

1.6.1 Código (Python local): bala que rebota

```
# pygame_bala_rebotando.py
import pygame
```

```
import sys

pygame.init()

WIDTH, HEIGHT = 800, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Bala rebotando con Pygame")

WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
RED = (255, 0, 0)

clock = pygame.time.Clock()

x = 50
y = HEIGHT // 2
vx = 0

while True:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_SPACE and vx == 0:
                vx = 8

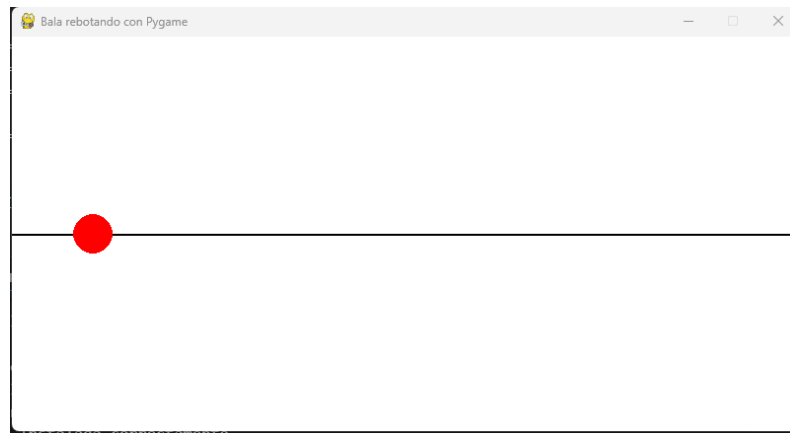
    x += vx

    if x > WIDTH - 20 or x < 20:
        vx = -vx

    screen.fill(WHITE)
    pygame.draw.line(screen, BLACK, (0, y), (WIDTH, y), 2)
    pygame.draw.circle(screen, RED, (int(x), y), 20)

    pygame.display.flip()
    clock.tick(60)
```

1.6.2 Ejecución (captura)



El programa abre una ventana donde se dibuja una línea horizontal y un círculo que representa una bala. Al pulsar la barra espaciadora, la bala comienza a moverse horizontalmente y, cuando alcanza los bordes de la pantalla, invierte su dirección, mostrando el patrón de rebote típico en simulaciones de colisiones simples.

1.7 MatManim (animaciones matemáticas)

La documentación de Manim Community describe la biblioteca como “an animation engine for explanatory math videos” que permite crear escenas programables a partir de objetos geométricos, texto y gráficos (Manim Community Developers, 2024). Este motor, originado en los videos de 3Blue1Brown, ofrece una API para definir transformaciones precisas en el tiempo, por lo que resulta especialmente útil para visualizar vectores, funciones y transformaciones lineales en un contexto educativo.

1.7.1 Código de escena (ejecución con manim)

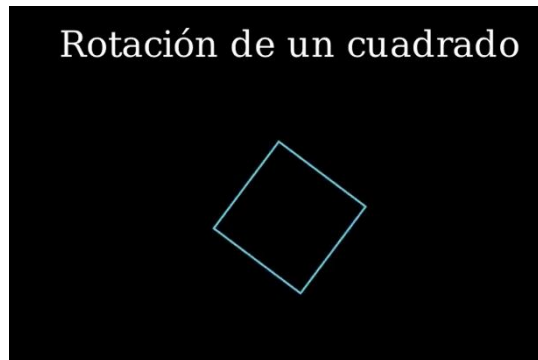
```
# manim_cuadrado_rotando.py
from manim import *

class CuadradoRotando(Scene):
    def construct(self):
        titulo = Text("Rotación de un cuadrado")
        self.play(Write(titulo))
        self.wait(1)
        self.play(titulo.animate.to_edge(UP))

        cuadrado = Square(side_length=2, color=BLUE)
        self.play(Create(cuadrado))
        self.wait(0.5)

        self.play(Rotate(cuadrado, angle=PI), run_time=2)
        self.play(cuadrado.animate.set_fill(RED, opacity=0.6), run_time=1)
        self.wait(1)
```

1.7.2 Ejecución (captura)



La escena comienza mostrando un título y luego dibuja un cuadrado en el centro de la pantalla, que se rota media vuelta y finalmente cambia de color y opacidad. Este ejemplo ilustra cómo Manim combina creación de objetos geométricos, transformaciones y efectos visuales para producir una animación matemática simple y controlada.

1.8 VPython (animación 3D sencilla)

La documentación de VPython señala que esta herramienta permite crear “3D animations and navigable scenes” utilizando objetos como esferas, cajas y vectores definidos en Python (VPython Developers, 2020). Los recursos educativos destacan su utilidad para enseñar física y matemática, ya que facilita visualizar trayectorias, fuerzas y campos en tres dimensiones con interacción de cámara en tiempo real.

1.8.1 Código (Python/Glowsript): órbita de un satélite

```
Web VPython 3.2
scene.width = 600
scene.height = 300
scene.center = vec(0,2,0)
scene.fov = .1
scene.background = color.white
scene.forward = vec(-0.3,-0.2,-1)

r = ring(radius=1, thickness=0.2, color=color.black, axis=vec(0,0,1))
c1 = cylinder(pos=vec(-1,0,0), axis=vec(2,0,0), radius=0.1)
c2 = c1.clone()
c2.rotate(angle=pi/3, axis=vec(0,0,1), origin=vec(0,0,0))
c3 = c1.clone()
c3.rotate(angle=-pi/3, axis=vec(0,0,1), origin=vec(0,0,0))
w = compound([r,c1,c2,c3])

g = group()
wheel1 = w.clone(pos=vec( 4,0, 0), group=g)
wheel2 = w.clone(pos=vec(-4,0, 0), group=g)
wheel3 = w.clone(pos=vec( 4,0,-4), group=g)
wheel4 = w.clone(pos=vec(-4,0,-4), group=g)
wheels = [wheel1, wheel2, wheel3, wheel4]
w.visible = False
```

```

body1 = box(group=g, pos=vec(0,0,-2), size=vec(8,2,3.5), color=color.red)
body2 = box(group=g, pos=vec(-1,2,-2), size=vec(3,2,3.5), color=color.red)
T = text(group=g, text='Car', pos=body2.pos+vec(-1,1,-0.6), depth=1,
color=color.green)

road = box(pos=vec(0,-1.3,-2), size=vec(30.6,0.2,7), color=color.gray(.95))
wall1 = box(pos=vec(-15.4,1.85,-2), size=vec(0.2,6.5,7), color=color.gray(.95))
wall2 = box(pos=vec(15.4,1.85,-2), size=vec(0.2,6.5,7), color=color.gray(.95))
g.pos = vec(-10,0,0)

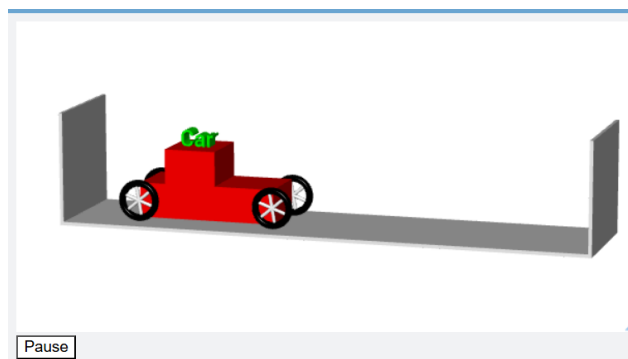
run = True
def running(b):
    global run
    run = not run
    if run: b.text = 'Pause'
    else: b.text = 'Run'

button(text='Pause', bind=running)

x = -10
dx = .05
dtheta = .5
while True:
    rate(100)
    if not run: continue
    g.pos.x = x
    for w in wheels:
        w.rotate(angle=-dtheta*dx, axis=vec(0,0,1))
    x += dx
    if x > 10 or x < -10:
        dx = -dx

```

1.8.2 Ejecución (captura)



En la demostración de Web VPython se construye un carro 3D compuesto por cajas, ruedas y texto, agrupado en un objeto group que se desplaza a lo largo de una carretera delimitada por paredes. Mediante un bucle animado y la función rate, el carro se mueve con velocidad constante, mientras las ruedas giran de forma sincronizada y el usuario puede pausar o reanudar la animación con un botón interactivo.

CAPÍTULO II

BLOQUE ANÁLISIS DE DATOS Y APRENDIZAJE AUTOMÁTICO

2.1 Concepto general del bloque

El bloque de análisis de datos y aprendizaje automático agrupa bibliotecas que abarcan estructuras de datos tabulares, algoritmos de cálculo científico, modelos de *machine learning* y marcos para visualización e interacción web. McKinney (2010) introduce pandas como una estructura de datos de alto nivel para análisis estadístico en Python, mientras que Virtanen et al. (2020) describen SciPy como una colección de algoritmos de cálculo científico integrados sobre NumPy. Por su parte, Pedregosa et al. (2011) presentan scikit-learn como un módulo que reúne una amplia gama de algoritmos de aprendizaje supervisado y no supervisado para problemas de tamaño medio. Waskom (2021) propone Seaborn como capa de visualización estadística sobre Matplotlib, y proyectos como Plotly, Bokeh y Streamlit se orientan a construir visualizaciones interactivas y aplicaciones web para explorar datos y modelos en tiempo real.

2.2 Pandas

McKinney (2010) presenta pandas como una biblioteca destinada a proporcionar estructuras de datos flexibles, tales como Series y DataFrame, para facilitar el análisis estadístico y el manejo de datos en Python. La documentación oficial indica que pandas está pensada para tareas prácticas de análisis y limpieza de datos en diversos dominios, apoyándose en NumPy para las operaciones numéricas de bajo nivel.

2.2.1 Código (Google Colab): resumen de ventas por categoría

```
# @title Resumen de ventas por categoría con pandas
import pandas as pd

datos = {
    "categoria": ["A", "A", "B", "B", "C", "C", "C"],
    "monto":     [100, 150, 200, 50, 300, 120, 80]
}

df = pd.DataFrame(datos)

print("Datos originales:")
print(df)

resumen = df.groupby("categoria")["monto"].agg(["count", "sum", "mean"])
print("\nResumen por categoría:")
print(resumen)
```


2.2.2 Ejecución (captura)

```
[Running] python -u "r:\UNIVERSIDAD\CICLO 10\COMPUTACIÓN GRÁFICA\codigos\Investigacion\pruebaPandas.py"
Datos originales:
  categoria  monto
0         A    100
1         A    150
2         B    200
3         B     50
4         C    300
5         C    120
6         C     80

Resumen por categoria:
  categoria  count  sum  mean
A         2    250  125.000000
B         2    250  125.000000
C         3    500  166.666667
```

El programa construye un conjunto de datos de ventas por categoría y los organiza en un DataFrame. A continuación agrupa los registros por categoría y calcula la cantidad de ventas, la suma total y el promedio del monto, mostrando cómo pandas facilita la agregación y el resumen de información.

2.3 NumPy (bloque de datos)

Desde la perspectiva del análisis de datos, Harris et al. (2020) señalan que NumPy proporciona arreglos multidimensionales y funciones matemáticas que constituyen la capa numérica sobre la que se construyen bibliotecas como pandas, SciPy y scikit-learn. Esta integración permite que múltiples herramientas compartan estructuras comunes y aprovechen optimizaciones internas para operaciones vectorizadas y matriciales.

2.3.1 Código (Google Colab): matriz y estadísticas

```
# @title Matriz y estadísticas básicas con NumPy
import numpy as np
```

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
```

```
print("Matriz A:\n", A)
print("Media global:", A.mean())
print("Desviación estándar:", A.std())
print("Suma por columnas:", A.sum(axis=0))
print("Suma por filas:", A.sum(axis=1))
```

2.3.2 Ejecución (captura)

```
[Running] python -u "r:\UNIVERSIDAD\CICLO 10\COMPUTACIÓN GRÁFICA\codigos\Investigacion\pruebaNumPy.py"
Matriz A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Media global: 5.0
Desviacion estandar: 2.581988897471611
Suma por columnas: [12 15 18]
Suma por filas: [ 6 15 24]
```

El script define una matriz de 3x3 y aplica funciones de NumPy para obtener la media, la desviación estándar y las sumas por columnas y filas. Este ejemplo ilustra cómo se pueden calcular estadísticas básicas de forma compacta utilizando operaciones vectorizadas.

2.4 Matplotlib (visualización de datos)

Hunter (2007) indica que Matplotlib proporciona funciones para generar diagramas de dispersión, gráficos de líneas, barras e histogramas, lo que la convierte en una herramienta clave para representar resultados numéricos. En el contexto de análisis de datos, se suele combinar con pandas y NumPy para visualizar patrones, tendencias y distribuciones de manera clara y configurable.

2.4.1 Código (Google Colab): dispersión con tendencia

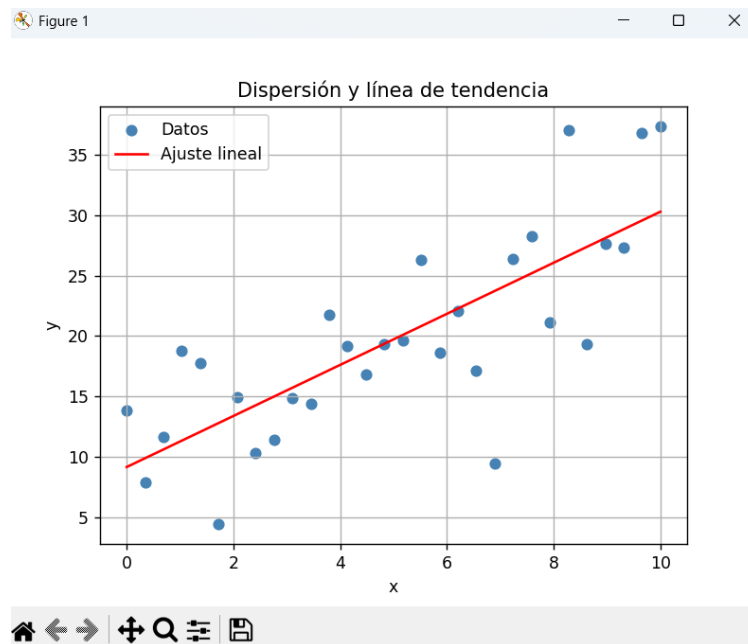
```
# @title Dispersión con línea de tendencia usando Matplotlib
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
x = np.linspace(0, 10, 30)
y = 2.5 * x + 5 + np.random.normal(0, 5, size=len(x))

coef = np.polyfit(x, y, 1)
y_fit = np.polyval(coef, x)

plt.scatter(x, y, label="Datos", color="steelblue")
plt.plot(x, y_fit, label="Ajuste lineal", color="red")
plt.title("Dispersión y línea de tendencia")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()
```

2.4.2 Ejecución (captura)



El programa genera datos con ruido que siguen aproximadamente una relación lineal, los muestra en un gráfico de dispersión y calcula una recta de ajuste usando una regresión polinómica de primer orden. La superposición de la línea roja permite visualizar la tendencia central de los datos frente a la dispersión observada.

2.5 Seborn

Waskom (2021) presenta Seaborn como una biblioteca de visualización estadística para Python basada en Matplotlib que ofrece una interfaz de alto nivel para explorar relaciones entre variables. En su artículo se destaca que Seaborn se integra con las estructuras de datos de pandas y proporciona funciones para trazar distribuciones, correlaciones y regresiones de manera más expresiva que la API básica de Matplotlib.

2.5.1 Código (Google Colab): matriz de correlación

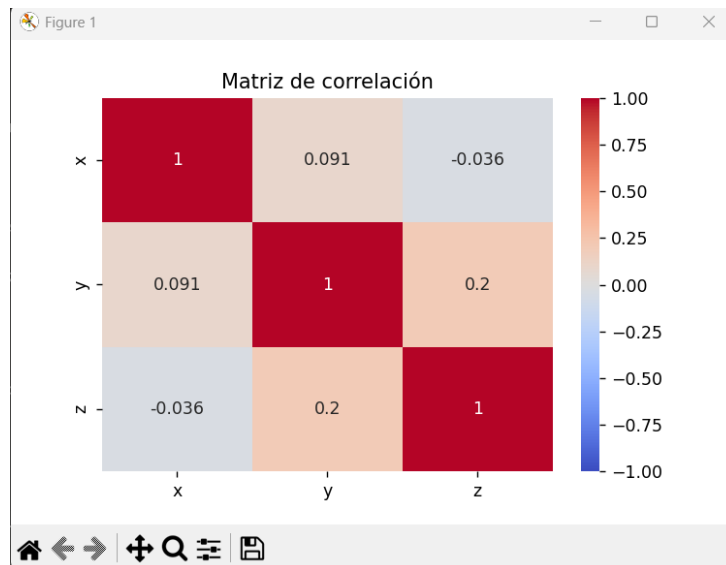
```
# @title Matriz de correlación con Seaborn
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
df = pd.DataFrame({
    "x": np.random.normal(0, 1, 100),
    "y": np.random.normal(5, 2, 100),
    "z": np.random.normal(-2, 3, 100)
})

corr = df.corr()
```

```
plt.figure(figsize=(6, 4))
sns.heatmap(corr, annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Matriz de correlación")
plt.show()
```

2.5.2 Ejecución (captura)



El código genera tres variables numéricas aleatorias, calcula la matriz de correlaciones y la representa mediante un mapa de calor. Los colores y las anotaciones numéricas permiten identificar de forma rápida la fuerza y el signo de las relaciones entre las variables.

2.6 Plotly (visualización interactiva en análisis de datos)

Para el análisis de datos, Plotly se utiliza como biblioteca de gráficos interactivos que permite construir dashboards y visualizaciones exploratorias directamente desde Python (Plotly Technologies Inc., 2015). Su capacidad para integrarse con pandas y ejecutarse en navegadores modernos facilita el desarrollo de interfaces de análisis visual sin necesidad de escribir código JavaScript.

2.6.1 Código (Google Colab): barras interactivas

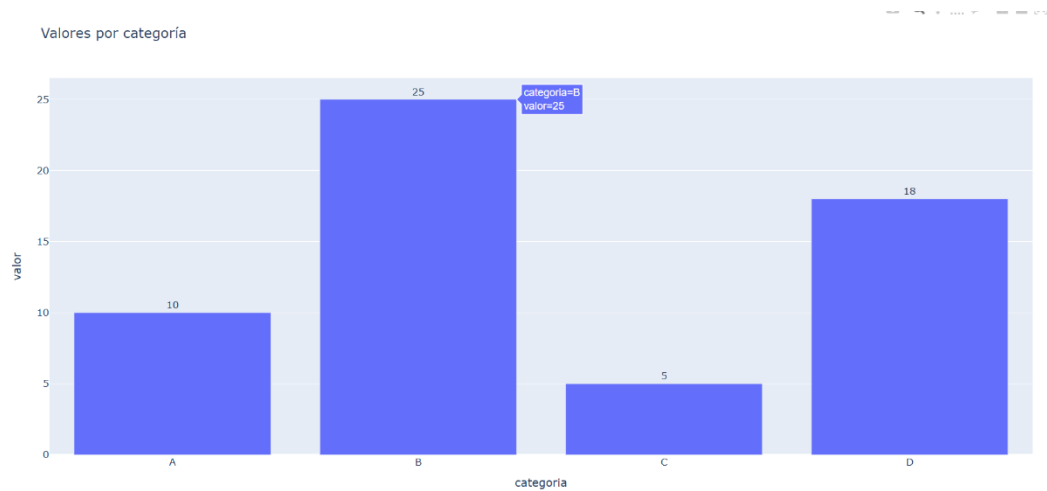
```
# @title Gráfico de barras interactivo con Plotly
import pandas as pd
import plotly.express as px

df = pd.DataFrame({
    "categoria": ["A", "B", "C", "D"],
    "valor": [10, 25, 5, 18]
})

fig = px.bar(df, x="categoria", y="valor",
             title="Valores por categoría",
             text="valor")
```

```
fig.update_traces(textposition="outside")
fig.show()
```

2.6.2 Ejecución (captura)



El script construye un conjunto simple de valores por categoría y lo representa en un gráfico de barras interactivo. El usuario puede explorar la figura mediante los controles estándar de Plotly, como el zoom y la descarga de la imagen, mientras que las etiquetas numéricas permiten interpretar con facilidad los valores asociados a cada barra.

2.7 SciPy

Virtanen et al. (2020) describen SciPy 1.0 como una biblioteca de código abierto que agrupa algoritmos de optimización, integración, interpolación, álgebra lineal, procesamiento de señales y estadística para el cálculo científico en Python. Los autores señalan que SciPy “has become a de facto standard for leveraging scientific algorithms in Python” gracias a su amplia comunidad y a su integración con NumPy.

2.7.1 Código (Google Colab): regresión lineal simple

```
# @title Ajuste lineal con SciPy
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

x = np.array([1, 2, 3, 4, 5])
y = np.array([1.2, 1.9, 3.1, 3.9, 5.1])

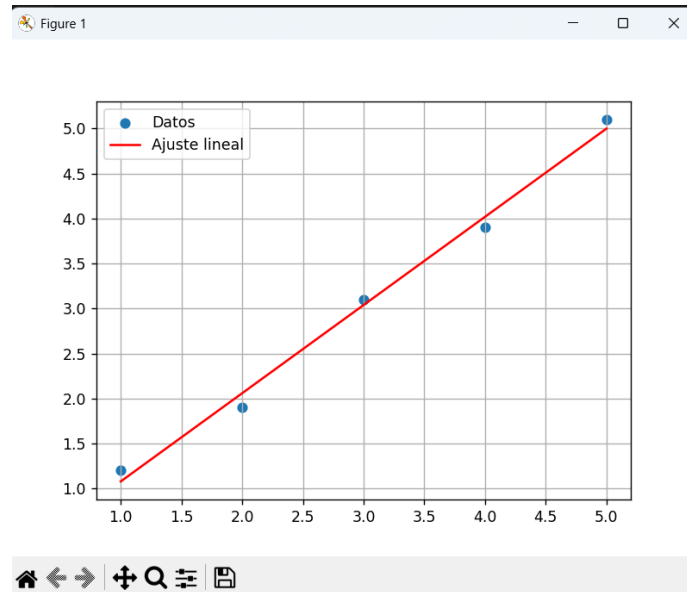
slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

print("Pendiente:", slope)
print("Intercepto:", intercept)
print("R^2:", r_value**2)

y_pred = slope * x + intercept
```

```
plt.scatter(x, y, label="Datos")
plt.plot(x, y_pred, color="red", label="Ajuste lineal")
plt.legend()
plt.grid(True)
plt.show()
```

2.7.2 Ejecución (captura)



El programa aplica una regresión lineal a cinco puntos, obteniendo la pendiente, el intercepto y el coeficiente de determinación. La gráfica resultante superpone la recta ajustada sobre los datos originales, lo que permite apreciar fácilmente la calidad del ajuste y la tendencia general de la relación entre las variables.

2.8 Scikit-learn

Pedregosa et al. (2011) definen scikit-learn como un módulo de Python que integra una amplia gama de algoritmos de aprendizaje supervisado y no supervisado para problemas de tamaño medio. En su artículo se destaca que la biblioteca ofrece una API coherente y herramientas para preprocesamiento, selección de modelos y evaluación, facilitando la aplicación de técnicas de *machine learning* a usuarios no especialistas

2.8.1 Código (Google Colab): clasificación binaria

```
# @title Clasificación binaria con scikit-learn
from sklearn.datasets import make_blobs
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

```
X, y = make_blobs(n_samples=200, centers=2,
                  random_state=0, cluster_std=2.0)
```

```

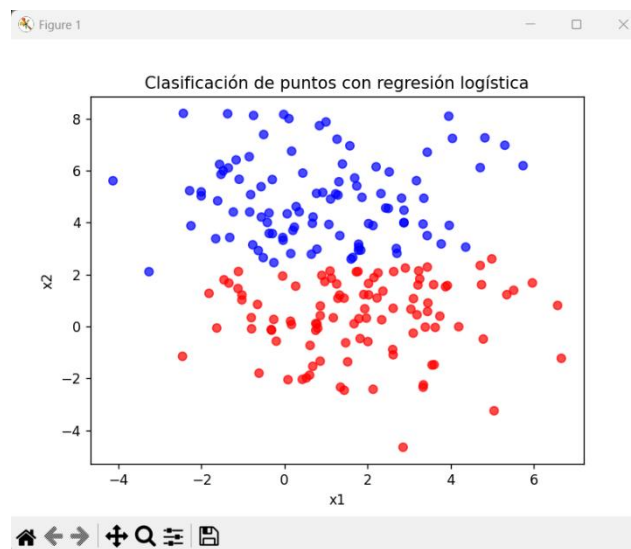
model = LogisticRegression()
model.fit(X, y)

y_pred = model.predict(X)
print("Exactitud:", accuracy_score(y, y_pred))

plt.scatter(X[:, 0], X[:, 1], c=y_pred, cmap="bwr", alpha=0.7)
plt.title("Clasificación de puntos con regresión logística")
plt.xlabel("x1")
plt.ylabel("x2")
plt.show()

```

2.8.2 Ejecución (captura)



El script genera dos grupos de puntos en el plano y entrena un modelo de regresión logística para clasificarlos. Después calcula la exactitud sobre el conjunto de entrenamiento y muestra un gráfico de dispersión donde los puntos aparecen coloreados según la clase predicha por el modelo.

2.9 Streamlit

Según la documentación oficial, Streamlit es una biblioteca de Python que permite crear aplicaciones web de datos con pocas líneas de código, dirigida principalmente a científicos de datos (Streamlit Inc., 2020). Los tutoriales muestran que la herramienta incluye componentes para cargar datos, crear gráficos y añadir controles interactivos sin necesidad de desarrollar la interfaz en HTML o JavaScript.

2.9.1 Código: mini dashboard de serie temporal

```

# streamlit_app.py
import streamlit as st
import pandas as pd
import numpy as np

```

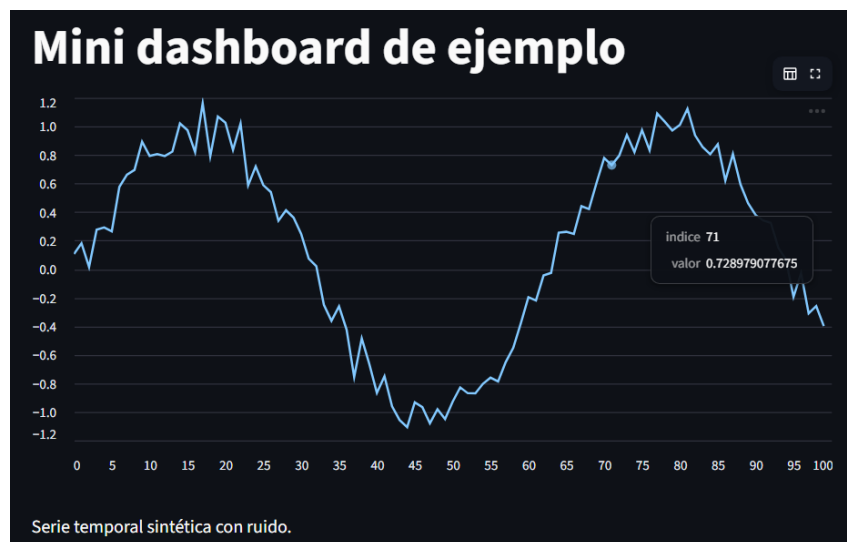
```
st.title("Mini dashboard de ejemplo")

x = np.arange(0, 100)
y = np.sin(x / 10) + np.random.normal(0, 0.1, size=len(x))

df = pd.DataFrame({"indice": x, "valor": y})

st.line_chart(df.set_index("indice"))
st.write("Serie temporal sintética con ruido.")
```

2.9.2 Ejecución (captura)



La aplicación genera una serie temporal sintética con una componente sinusoidal y ruido, y la muestra en un gráfico de líneas interactivo. Al recargar la ejecución se obtienen nuevas realizaciones de la señal, lo que permite explorar la variabilidad del proceso de forma visual.

2.10 Bokeh

La documentación de Bokeh describe esta biblioteca como una solución para crear visualizaciones interactivas en el navegador a partir de datos en Python (Bokeh Development Team, 2018). Se enfatiza que Bokeh integra componentes para construir gráficos, paneles y aplicaciones web, trabajando tanto en cuadernos Jupyter como en archivos HTML autosuficientes o servidores específicos

2.10.1 Código (Google Colab / HTML): línea interactiva

```
# @title Gráfico de línea interactivo con Bokeh
from bokeh.plotting import figure, output_file, show

output_file("linea_bokeh.html")

p = figure(title="Ejemplo de línea con Bokeh",
           x_axis_label="x",
```



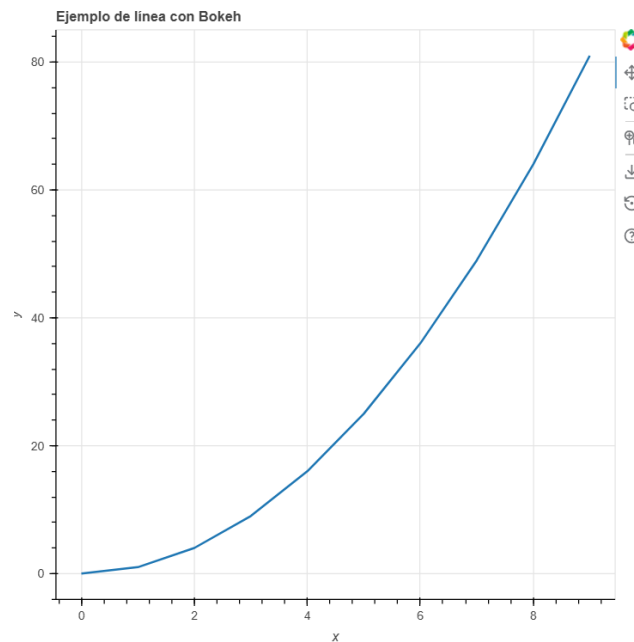
```
y_axis_label="y")

x = list(range(0, 10))
y = [i**2 for i in x]

p.line(x, y, line_width=2)

show(p)
```

2.10.2 Ejecución (captura)



El programa genera un gráfico de la función cuadrática y lo guarda en un archivo HTML que se abre en el navegador. La interfaz de Bokeh permite aplicar zoom, desplazamiento y otras interacciones sobre la curva, lo que muestra su capacidad para crear visualizaciones web interactivas a partir de código Python.

CONCLUSIONES

Python ofrece, sin lugar a dudas, un conjunto robusto de bibliotecas maduras y bien documentadas. Estas integran de forma inequívoca la computación gráfica, la animación, el análisis de datos y el aprendizaje automático.

Para gráficos y animación, tienes Matplotlib, FuncAnimation, Plotly, Pygame, Manim, VPython... y la lista continúa. Puedes crear desde sencillos gráficos 2D hasta animaciones súper interactivas e incluso simulaciones 3D para el aprendizaje. NumPy te respalda en todo lo relacionado con números. Básicamente, si quieres mostrar funciones, rutas o escenas 3D, estas herramientas lo hacen posible. ¡También son muy útiles para enseñar física y matemáticas!

Además, está el análisis de datos y el aprendizaje automático. Pandas, SciPy, Scikit-learn, Seaborn, Streamlit, Bokeh: lo cubren todo. Desde la obtención y limpieza de datos hasta la creación de modelos y paneles interactivos. Hemos visto cómo estas herramientas se integran a la perfección, haciendo que tu flujo de trabajo sea fluido y repetible. Puedes transformar datos, crear modelos y compartir tus hallazgos con increíbles paneles o aplicaciones web.

Sinceramente, el ecosistema científico de Python es una opción obvia para cualquier proyecto de ingeniería o informática que requiera visualización, simulación y análisis de datos. Es así de bueno.

BIBLIOGRAFÍA

- Bokeh Development Team. (2018). Bokeh documentation. Bokeh. <https://docs.bokeh.org>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Manim Community Developers. (2024). Manim Community: An animation engine for explanatory math videos [Software]. Manim Community. <https://www.manim.community>
- McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference (SciPy 2010)* (pp. 51–56). <https://conference.scipy.org/proceedings/scipy2010/mckinney.html>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. <http://www.jmlr.org/papers/v12/pedregosa11a.html>
- Plotly Technologies Inc. (2015). Plotly: Python open source graphing library [Software]. Plotly. <https://plotly.com/python>
- Pygame Community. (2024). Pygame documentation (Version 2.6.0) [Computer software]. Pygame. <https://www.pygame.org/docs>
- Streamlit Inc. (2020). Create an app – Streamlit documentation [Tutorial]. Streamlit. <https://docs.streamlit.io/get-started/tutorials/create-an-app>
- The Bokeh Team. (2018). Bokeh: An interactive visualization library for modern web browsers [Computer software]. Bokeh. <https://bokeh.org>
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., ... van der Walt, S. J. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- VPython Developers. (2020). VPython: 3D animations and navigable scenes in Python [Computer software]. VPython. <https://vpython.org>